To sort the result, use `orderBy()`:

```
$users = DB::table('users')
           ->orderBy('created_at', 'desc')
           ->get();
```

To limit the number of results, use `limit()`:

```
$users = DB::table('users')->limit(10)->get();
```

To skip a number of records (for pagination), use `offset()`:

```
$users = DB::table('users')->offset(10)->limit(10)->get();
```

11.2.4 Aggregates

Query Builder supports SQL aggregate functions like `count`, `sum`, `avg`, `min`, and `max`:

```
$totalUsers = DB::table('users')->count();
$averageAge = DB::table('users')->avg('age');
$maxSalary = DB::table('employees')->max('salary');
```

11.2.5 Joins

You can perform inner joins using the `join()` method:

```
$orders = DB::table('orders')
            ->join('users', 'orders.user_id', '=', 'users.id')
            ->select('orders.*', 'users.name')
            ->get();
```

Laravel also supports left joins via `leftJoin()`, cross joins with `crossJoin()`, and even advanced join conditions using closures.

11.3 Insert, Update, and Delete

In addition to retrieving records, Laravel's Query Builder makes it simple to insert, update, and delete data in the database. These operations follow a consistent and expressive syntax, which improves code readability and maintainability.

11.3.1 Inserting Records

To insert a new record into a table, use the `insert()` method. This method accepts an associative array where the keys are the column names and the values are the values to insert.